Enabling reasoning mechanisms in Small Language Models for compiler optimizations

Emil Fakhretdinov
240897244
Lev Mukhanov
MSc Artificial Intelligence

***Abstract*—Peephole optimizations are small compiler transformations that improve performance while preserving semantics. Traditional rule-based approaches require expert effort and are hard to scale. This study examines whether Small Language Models (SLMs, ≤3B parameters) can perform correct LLVM Intermediate Representation (IR) peephole optimizations without fine-tuning, using reasoning-enhanced prompting. We evaluate general-purpose and code-specialized SLMs in single-model and dual-model (reasoner–transformer) setups, with reasoning provided via Chain-of-Thought (CoT) prompting or larger external models. All outputs are verified with Alive-tv, using Overall Transformation Rate (OTR), Correct Transformation Rate (CTR), and No-Transformation Rate (NTR). Code-specialized SLMs outperform general-purpose ones in single-model tests (CTR up to 47.4%), while the best results come from dual-model prompting–GPT-o3-mini reasoning with Qwen2.5-Coder-3B, achieving an OTR of 76.4% and CTR of 68.6%. CoT benefits smaller models but may reduce performance in 3B parameter models. These findings suggest reasoning-enhanced prompting offers a scalable, low-cost alternative to large-model fine-tuning for compiler optimization.**

Keywords—compiler optimization, LLVM IR, peephole optimization, small language models, Chain-of-Thought.

I. Introduction

Modern compiler design relies on continual optimization to improve execution speed, reduce memory usage, and remove redundant computations. Peephole optimizations—small, localized instruction simplifications—are both safe and effective (Bhatt and Bhadka, 2013) but traditionally require expert-crafted rules, making them time-consuming and hard to scale. With the advent of Large Language Models (LLMs) such as ChatGPT (OpenAI, 2023), DeepSeek (DeepSeek-AI, 2024), and Claude (Anthropic, 2025) has increased interest in automating compiler optimization by leveraging their pattern recognition and code generation abilities (Cummins et al., 2023). Built on the Transformer architecture (Vaswani et al., 2017), LLMs efficiently process long dependencies and parallel computations, making them well-suited for code tasks (Feng et al., 2020; Fried et al., 2022) and potentially improving compiler development (Hosseini and Dolan-Gavitt, 2022). However, LLMs can produce hallucinations, omit steps, or introduce semantic errors (Agarwal et al., 2024; Liu et al., 2024), with such issues more frequent in Small Language Models (SLMs, <3B parameters) (EvalPlus, 2025). Unlike natural language, where small errors may be tolerable, in compilers even one incorrect instruction can break program correctness (Lattner and Adve, 2004). To mitigate this, **reasoning mechanisms** have been proposed to improve step-by-step logical consistency (Wei et al., 2022; Yao et al., 2023), encouraging models to analyse their reasoning before generating outputs. We apply this to compiler pipelines, where reasoning can connect high-level optimization goals with low-level transformations, crucial in peephole optimization to identify inefficiencies while preserving semantics. Later chapters show that adding reasoning improves SLM performance on peephole optimization tasks compared to instruction-based prompting and even larger fine-tuned models without reasoning.

II. Problem statement

Previous research on compiler optimization with machine learning and Language models has targeted superoptimization (Mankowitz *et al*., 2023), instruction scheduling and inlining (Trofin *et al*., 2021) and heuristic tuning (VenkataKeerthy *et al*., 2023), often using training models from scratch, reinforcement learning or fine-tuning LLMs for specific compiler passes. While effective, these methods are resource-intensive and struggle to generalize without retraining.

Prompting-based techniques offer a more lightweight alternative, for adapting models to new tasks, but Small Language models (SLMs), which are more practical for deployment, often struggle with correctness in code tasks. We focus on peephole optimization as it operates on small instruction windows, fitting within SLM context limits (4k-32k tokens) and (4k-8k tokens) generation limits (Abdin *et al*., 2024), (Qwen, 2024) and avoiding the need for window management, unlike longer-sequence tasks where LLM performance degrades.

This study investigates whether Small Language Models (SLMs, ≤3B parameters) can perform semantics-preserving peephole transformations on LLVM Internal Representation (IR) code without fine-tuning. We aim to answer the following research questions:

- Can reasoning prompting techniques, such as Chain-of-Thought (CoT) (Kojima *et al*., 2022), enable SLMs to apply correct and legal LLVM-IR peephole optimizations without any additional training?
- How does the performance of instruction-tuned models compare to code-specialized models in this task?
- Can reasoning generated by Large Language Models be effectively used to enhance the optimization performance of Small Language Models through mixed-model prompting techniques?

A positive result would show that reasoning-based prompting can be a lightweight, fine-tuning-free alternative, and that large models can guide smaller ones in hybrid workflows, enabling effective SLM-assisted compilers without high deployment costs.

III. Contributions

This thesis investigates the use of reasoning-enhanced prompting strategies to enable Small Language Models to perform legal and semantically correct peephole optimizations

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

over LLVM IR. Without any model fine-tuning, the proposed approach achieves **68.6% accuracy** in producing verified IR transformations. The key contributions are:

- **Verified evaluation pipeline for LLVM-IR peephole optimization.** We design a reasoning driven evaluation pipeline for LLVM IR transformation that combines instruction-style examples, few-shot context and Chain-of-Thought prompting in SLM with Alive-tv translation validation and introduce task-specific metrics - OTR, CTR, NTR to quantify legal, semantic preserving transformations.
- **A comparative study of SLM families.** We compare general-purpose SLMs (Qwen2.5-Instruct (Hugging Face, 2025a)), code-specialized SLMs (Qwen2.5-Coder-Instruct (Hugging Face, 2025b)) and distilled-reasoning variant (Shridhar, Stolfo and Sachan, 2022), (Hugging Face, 2025c). Number of parameters from 1.5B to 3B. Showing how prompt style and models size jointly affect transformation quality. Notably, CoT helps tiny models but not larger ones (Qwen2.5-Coder-1.5B **CTR +9.5 pp**, OTR +17.2 pp. Qwen2.5-Coder-3B CTR -0.3 pp, OTR -12.7 pp)
- **Mixed-model (dual-stage) prompting that outperforms single-model baselines.** We show that separating reasoning to a stronger model and IR generation to a code-specialized SLM provides the best results, inspired by Draft-and-Verify (Zhang *et al.*, 2024). **GPT-o3-mini** and **Qwen2.5-Coder-3B** achieves **OTR = 76.4% and CTR = 68.6%**, exceeding all single-model configurations without any fine-tuning.
- **Negative results and ablations documented.** We evaluate two exploratory directions and explain why they were not retained: PPO fine-tuning proved to be impractical with Alive-tv non-differentiable reward. Automatic Prompt Engineering (APE) underperformed (20% Alive-tv pass) versus our hand-crafted prompts.
- **Integration of Alive-tv** (AliveToolkit, 2025), a formal verification tool to automatically validate the semantic equivalence of model-generated transformations and to quantify optimization correctness.

This research aims to bridge the gap between the strengths of Small Language Models and the strict requirements of compiler optimization, offering a scalable and verifiable approach to reasoning-aware code transformation.

IV. Background & Related Work

A. LLVM IR and Peephole Optimization

LLVM (Low-level Virtual Machine) is a widely used compiler framework that uses Intermediate Representation (IR) to provide platform-independent optimization and code generation (Lattner and Adve, 2004). LLVM IR is a structured, strongly typed language based on Static Single Assignment (SSA) form, which makes it well suited for precise transformations at different stages of compilation (LLVM.org, 2025). One of the important categories of such transformations is peephole optimization, small local changes applied to short sequences of commands. These include tasks such as constant folding, dead code elimination and algebraic simplifications. Most of these changes are implemented in the *-passes=instcombine* pipeline, which combines dozens of canonical peephole patterns into a single pass.

B. Alive-tv as a verification tool

While neural models can be effective in generation or transforming code, ensuring semantic correctness remains a critical requirement. Alive-tv extends the Alive2 framework (AliveToolkit, 2025). It uses SMT solvers to check whether the original and optimized IRs are semantically equivalent (Lopes *et al.*, 2021). We integrate Alive-tv into our pipeline to scope every model-generated transformations for correctness and reliability.

C. Reasoning Mechanisms in Language Models

1) Chain-of-Thought (CoT): CoT prompting asks model to write its intermediate reasoning steps before producting the final answer. The technique, first validated by *(Wei* et al*., 2022)*, has since been strengthened by Self-Consistency decoding, which samples multiple CoT paths and chooses the majority answer, raising accuracy on math tasks *(Wang* et al*., 2022)*. Program-of-Thoughts prompting goest further, by separating symbolic computation from textual reasoning by letting the model to generate small code snippets that it can execute, which provides advantages in numerical reasoning-based tasks *(Chen* et al*., 2022)*.

2) Tree-of-Thought (ToT): ToT prompting extends the same idea, the model develops several reasoning branches in parallel, turning them into searchable tree and using best-first and breadth-first algorithms to explore alternative solutions. *(Yao* et al*., 2023)*

D. Distilling reasoning into Small Language Models

Recent work such as CodePLAN (Sun *et al.*, 2024) investigates how reasoning capabilities can be distilled into small language models to improve code generation. CodePLAN proposes a multi-task learning framework, where a SLM is trained to produce both code and corresponding reasoning steps, referred to as "solution plans", using supervision from LLM, see Fig. 1 for more details. This strategy leads to a significant performance gains on benchmarks such as APPS (Hendrycks *et al.*, 2021) and MBPP (Austin *et al.*, 2021), demonstrating that small models can benefit from structured reasoning, even without relying on LLMs during inference. Inspired by this line of research, we included an evaluation of DeepSeek-R1-Distill-Qwen-1.5B (Hugging Face, 2025c), a distilled variant of DeepSeek-R1 (DeepSeek-AI, 2025) designed to retain reasoning capabilities in a compact 1.5 billion parameter form.

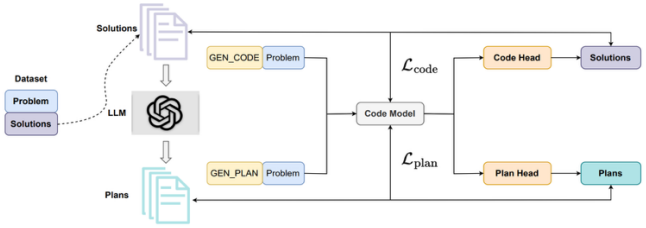


Fig. 1. Framework for the training phase of CodePLAN. Reprinted from (Sun *et al.*, 2024).

E. Reinforcement learning for reasoning

Beyond prompt engineering, reinforcement learning (RL) techniques are widely used to improve reasoning in language models. RL gives a model a reward signal, guiding it toward outputs that satisfy task-specific objectives. The most common algorithm in language model tuning is Proximal Policy Optimization (PPO) (Schulman *et al*., 2017)**,** which stabilizes gradient updates by clipping the policy ratio. PPO is a basis of Reinforcement Learning with Human Feedback (RLHF) as applied in InstructGPT (Ouyang *et al*., 2022). It is often combined with techniques like CoT prompting or supervision from larger models to guide learning. By integrating reinforcement learning researchers aim to improve the consistency, correctness and reliability of model generated reasoning (Havrilla *et al*., 2024).

F. Language Models within Compiler Optimization

The Meta AI paper presents a 7B-parameter model trained to assist compiler optimization by predicting effective sequences of LLVM optimization passes from unoptimized IR (Cummins *et al*., 2023). Instead of generating code or performing reasoning, it outputs a pass list applied by LLVM, leaving correctness to the compiler. This achieves a 3% average instruction reduction and aligns with earlier phase-ordering methods using deep RL (Haj-Ali *et al*., 2019). Fig. 2 illustrates the pipeline proposed by (Cummins *et al*., 2023).

An alternative direction is explored by (Fang and Mukhanov, 2024), where a fine-tuned LLM is tasked with generating optimized code directly. A 7 billion parameter LLaMA2 model (Touvron *et al*., 2023) is trained on peephole optimization of AArch64 basic blocks and evaluated for syntactic and semantic correctness. Reported metrics include 64% syntactic accuracy and 51.6% IO accuracy, with GPT-o1 achieving significantly higher correctness through chain-of-thought. Unlike the pass-sequence approach, this work shifts responsibility for correctness to the model itself, but at a lower, target-specific abstraction level. In our study, although the approaches are not directly comparable, we use syntactic validity and verified transformations as common criteria, which allows for a meaningful comparison despite differences in representation level.

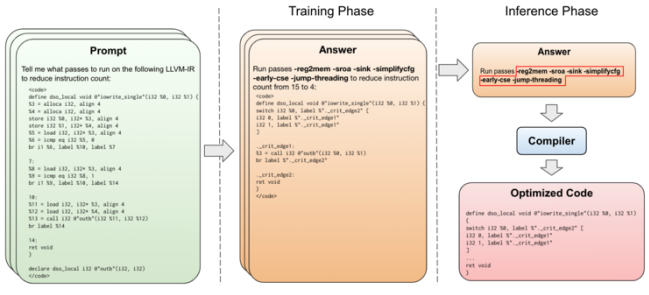


Fig. 2. Overview of the approach, showing the model input (prompt) and output (answer) during training and inference. The prompt contains unoptimized code. The answer contains an optimization pass list, instruction counts, and the optimized code. During inference, only the pass list is generated and fed into the compiler to ensure correctness. Reprinted from (Cummins *et al*., 2023).

G. Automated prompt engineering

Recent work shows that language models can act as a meta-prompt engineers. Autonomously proposing and ranking prompts that improve performance on downstream tasks. The Automatic Prompt Engineer (APE) framework lets a model iteratively generate, evaluate and refine prompts through a self-contained feedback loop (Zhou *et al*., 2022). Building on these findings, we conducted exploratory experiments in applying LLM-generated prompts to LLVM IR transformation tasks. Although these studies are secondary to the main study, they investigate whether self-generated prompting strategies can further improve the quality of reasoning without using manually created examples.

V. Methodology

A. Language models evaluated

We benchmark four open-sourced Qwen2.5 SLMs (Hugging Face, 2025b), a distilled reasoning model from DeepSeek-AI (Hugging Face, 2025c), and large model such as GPT-4.1-mini and GPT-o3-mini accessed via the OpenAI API (OpenAI, 2025) for dual-model strategy. Table I summarises their parameters, training focus, and roles in our pipelines.

TABLE I. Language models evaluated

Model	Model characteristics	
	Training focus	***Role in pipeline***
Qwen2.5-1.5B	General instruction following	**Single-stage** evaluation
Qwen2.5-3B	General instruction following	**Single-stage** evaluation and **reasoner** in dual-model strategy
Qwen2.5-Coder-1.5B	Multilingual code corpus	**Single-stage** evaluation
Qwen2.5-Coder-3B	Multilingual code corpus	**Single-stage** evaluation and **IR generator** in dual-model strategy
DeepSeek-R1-Distill-Qwen-1.5B	Distilled reasoning from DeepSeek-R1	Compact reasoner in **single-stage** evaluation
GPT-4.1-mini	General instruction following	External **reasoner** in dual-model strategy
GPT-o3-mini	General instruction following	External **reasoner** in dual-model strategy

[a.] B in model name indicates for number of parameters in billions.

Single stage indicates configurations where the same model handles both reasoning and code generation. **Reasoner** model produces step-by-step CoT, which is forwarded to the IR generator. **IR generator** means the model produces the optimized LLVM IR after receiving the reasoning trace.

B. Dataset

The dataset used in this study consists of paired samples of unoptimized and optimized LLVM Intermediate Representation (IR) code, focusing on peephole transformations. To generate these pairs, source code was first compiled into LLVM IR. Unoptimized IR was obtained by invoking the *opt* tools with *-O0* flag, while optimized IR was produced by using *opt -instcombine* command, which applies a range of optimizations. These IRs were then aligned and extracted. Each data sample includes, **non_optimized**: the original LLVM IR before any transformations and **LLVM_instcombine**: the target IR after valid *instcombine* optimizations.

C. Preprocessing

To ensure data consistency and compatibility with model inputs, outputs and Alive-tv check, several preprocessing steps were applied. IRs exceeding 2000 tokens were discarded to ensure they fit within the context and generation limits of small language models.

- Remove markdown formatting, such as "```llvm" and other unnecessary symbols.
- Normalize whitespace and indentation to keep all samples consistent.
- Check syntax to make sure every input is valid LLVM IR and can be used with the Alive-tv.

D. Prompting strategies

Designing effective prompts was a critical part of this study, particularly when using small language models with limited capacity for generalization. Main prompting formats that were used:

1) Basic Prompting: Is a minimal deterministic instruction. Where the model is asked to "Optimize the following IR:" followed by the code snippet. The full system and user prompt are shown in Appendix A.1

2) Few-Shot Prompting: In this format, the model is given 3 examples of unoptimized and optimized IR pairs directly in the prompt. Each example was followed by a target instruction: "Optimize the following IR:". This format relied on analogical reasoning and pattern matching *(Brown* et al*., 2020)*. An example is provided in Appendix A.2

3) Chain-of-Thought Prompting: This strategy requires the model to explicitly explain its transformation steps under a "Plan" heading before generating optimized IR under "Final IR" heading. The prompt template is provied in Appendix A.3.

4) Dual-model Configuration: A notable setup explored in this thesis was a dual-model reasoning pipeline, which separates reasoning and transformation into two stages. The prompt template is provided in Appendix A.4.

a) Reasoning model: A language model (Qwen2.5-3B, GPT-4o-mini and GPT-o3-mini) is prompted with the original IR and tasked with generating step-by-step explanation or justification of the optimization process.

b) Transformation model: The generated reasoning is then passed as context to a second model (Qwen2.5-Coder), which is responsible for generating the optimized IR based on that reasoning. Fig. 3 illustrates the pipeline example.

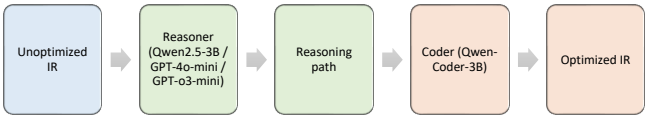


Fig. 3. Dual-model pipeline showing the reasoning stage, where an instruction-based model generates an optimization path from unoptimized LLVM IR and transformation stage of the dual-model pipeline, where a code-specialized model produces optimized IR based on the generated reasoning path.

This modular configuration allows the use of more capable general-purpose models for reasoning, while leveraging smaller, code-specialized models for final transformation. It also enables a clearer separation between logic and execution, supporting better interpretability and potential reuse of reasoning patterns. For single-model experiments, a few-shot prompt serves as the baseline, because few-shot significantly improves Alive-tv pass rate. In dual-model configurations we also report a basic zero-shot prompt, since the reasoning load is delegated to the separate reasoner model. Automatic Prompt Engineering (APE) was evaluated but not retained (details in Section V-F).

E. Evalutation Pipeline

The evaluation process was designed to assess both the syntactic validity and semantic correctness of model generated LLVM IR transformations.

1) Input and Generation setup: Each unoptimized IR sample was prompted using Basic, Few-shot or Chain-of-Thought (CoT) configurations. Generation was performed via sampling with temperature of 0.7, top-p = 0.95 and a 4096 token limit. The moderately high temperature encouraged exploration of diverse yet valid optimization paths while preserving a bias toward high-probability continuations. All experiments were conducted on a single NVIDIA RTX 4090 GPU. Due to the memory demands assosiated with long-context reasoning-partucularly in CoT and dual-model configuration. Each IR sample was processed individually without batching. This ensured consistent model behavior and prevented out-of-memory errors.

2) Verification stage (Alive-tv): To ensure semantic correctness, all generated IR transformations were formally verified using Alive-tv. Alive-tv leverages SMT solvers to prove the functional equivalence between the original and optimized IR. Fig. 4 illustrates the verification pipeline. The output of Alive-tv is a structured dictionary indicating one of the following: **success** (transformation is functionally equivalent to the original), **fail** (a semantic mismatch or invalid IR was detected), **timeout** (The SMT solvers exceeded a 60-second execution limit, these cases were treated as failures).

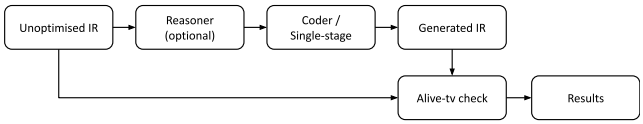


Fig. 4. Verification pipeline for LLVM IR transformations using Alive-tv. Optinal reasoning precedes transformation. All outputs are verified for semantic equivalence using SMT solvers.

3) Metrics: Three primary metrics were used to evaluate model performance in generating correct LLVM IR transformations.

- **Overall Transformation Rate (%) (OTR)**: The total percentage of outputs that passed Alive-tv verification, regardless of whether an actual transformation occurred. This includes both correct transformations and no-change passes. Providing view of the model's ability to preserve semantics.
- **Correct Transformation Rate (%) (CTR)**: This metric reflects the percentage of model outputs that both (i) passed Alive-tv verification and (ii) differed from the original unoptimized IR. It captures the rate of successful, semantic-preserving transformations,

cases where the model performed a valid optimization.

- **No transformation Rate (%) (NTR)**: The percentage of model outputs that passed Alive-tv, but were identical to the original unoptimized IR. This indicates how often the model avoided making any changes to ensure correctness, rather that attempting an actual optimization

F. Exploratory Baselines and Ablations

1) PPO Fine-tuning: We tried fine-tuning Qwen2.5-Coder-1.5B with Proximal Policy Optimization (PPO) *(Schulman* et al*., 2017)*, rewarding Alive-tv verified transformations and bonus rewards for reducing the number of IR instructions. However, current distributed-training implementation of TRL library *(Hugging Face, 2025d)* requires a learnable reward model, bot an external function. Because our reward is an external equivalence check (Alive-tv), it cannot be expressed as a differentiable model. This incompatibility made PPO fine-tuning unsuitable for our pipeline and the approach was abandoned.

2) Automatic prompt Engineering (APE): We tested the Automatic Prompt Engineer (APE) by giving GPT-o3-mini 25k IR pairs to generate and mutate prompts, ranking them with Qwen2.5-Coder-1.5B and Alive-tv. The best achieved only a 20% pass rate, below the hand-crafted CoT template, as most lacked reasoning ability. Example in Appendix A.4.

These negative results confirm our decision to rely on CoT-style prompts using SLM that provide higher correctness with lower compute.

VI. Results

A. Overview

Table II reports the performance of all evaluated configurations in terms of the three metrics introduced in section V. We evaluated each configuration on a sample of 1,000 LLVM IR pairs from the filtered dataset (≤ 2000 tokens) due to time and computational constraints.

- **Overall Transformation Rate (OTR)** – Alive-tv pass rate over all outputs.
- **Correct Transformation rate (CTR)** – Alive-tv pass rate where the IR changed.
- **No Transformation Rate (NTR)** – Alive-tv pass rate with no change in IR.

In single-model configurations same model perform both reasoning and IR generation. Dual-model configurations are denoted *Reasoner* → *Transformer*, where the first model produces the reasoning trace and the second produces the optimized IR. The grouped performance for all configurations is summarized in Fig. 5, showing OTR, CTR and NTR for each model-prompting strategy. For reference, the dashed red line indicates the 64% CTR reported for the fine-tuned LLaMA2-7B (Fang and Mukhanov, 2024).

TABLE II. RESULTS

ID	Configuration	Metrics			
		Prompting strategy	*OTR %*	*CTR %*	*NTR %*
1	Qwen2.5-1.5B	FS	67.2	12.6	54.6
2	Qwen2.5-1.5B	FS+CoT	33.7	14.9	18.8
3	Qwen2.5-3B	FS	45.5	41.1	4.4
4	Qwen2.5-3B	FS+CoT	43.7	41.5	2.2
5	Qwen2.5-Coder-1.5B	FS	46.5	10.8	35.7
6	Qwen2.5-Coder-1.5B	FS+CoT	63.7	20.3	43.4
7	Qwen2.5-Coder-3B	FS	73.2	47.4	25.8
8	Qwen2.5-Coder-3B	FS+CoT	60.5	47.1	13.4
9	DeepSeek-R1-Distill-Qwen-1.5B(internal reasoning model)	FS	16.7	13.8	2.9
10	Qwen2.5-3B→Qwen2.5-Coder-3B	BP	66.4	52.3	14.1
11	Qwen2.5-3B→Qwen2.5-Coder-3B	FS	70.9	58.7	12.2
12	GPT-4.1-mini→Qwen2.5-Coder-3B	BP	68.8	66.9	1.9
13	GPT-o3-mini→Qwen2.5-Coder-3B	BP	**76.4**	**68.6**	7.8

b. FS = few-shot, CoT = Chain-of-Thought, BP = basic prompt. N = 1000

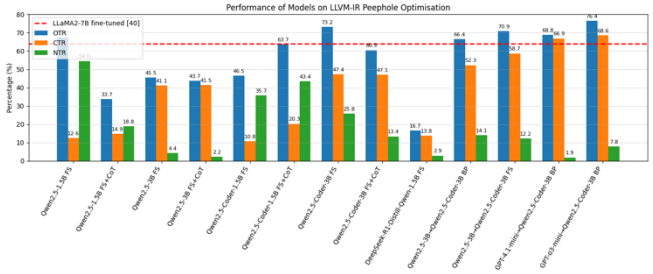


Fig. 5. Grouped results for OTR, CTR and NTR across all configurations. Dashed red line shows 64% from fine-tuned LLaMA2 on AArch64 peephole optimization (Fang and Mukhanov, 2024).

In addition to correctness-based metrics, we measured the relative change in LLVM IR instruction count produced by each configuration (Fig. 6). This was calculated as a percentage difference between the original and optimized IR, with negative values indicating a decrease.

- Mean change among successful transformations only (Alive-tv successful pass and output IR that differed from the input IR)
- Mean change over all inputs. Including no-change and failed cases, to evaluate the overall effect of the pipeline.

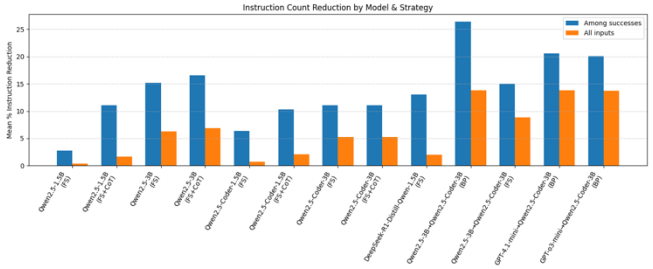


Fig. 6. Mean percentage reduction in LLVM IR instruction count by model and prompting strategy, reported both among successful transformations (Alive-tv–verified and different from the original) and across all evaluated inputs.

In most configurations, the verified transformations led to a reduction in instruction count. The largest decrease was seen in the dual-model setup. The Qwen2.5-Coder-3B → Qwen2.5-Coder-3B pipeline achieved a **26.43%** mean reduction for successful transformations, with a **13.82%** overall reduction across all inputs. Detailed results with the breakdown for each model are shown in Table III.

TABLE III. RELATIVE CHANGE IN LLVM IR INSTRUCTION COUNT AFTER OPTIMIZATION

ID	Configuration	Metrics		
		Prompting strategy	*Mean % Δ (successes)*	*Mean % Δ (all inputs)*
1	Qwen2.5-1.5B	FS	2.73	0.34
2	Qwen2.5-1.5B	FS+CoT	11.09	1.65
3	Qwen2.5-3B	FS	15.18	6.24
4	Qwen2.5-3B	FS+CoT	16.57	6.87
5	Qwen2.5-Coder-1.5B	FS	6.38	0.68
6	Qwen2.5-Coder-1.5B	FS+CoT	10.31	2.06
7	Qwen2.5-Coder-3B	FS	11.09	5.26
8	Qwen2.5-Coder-3B	FS+CoT	11.07	5.22
9	DeepSeek-R1-Distill-Qwen-1.5B(internal reasoning model)	FS	13.01	1.95
10	Qwen2.5-3B→Qwen2.5-Coder-3B	BP	**26.43**	**13.82**
11	Qwen2.5-3B→Qwen2.5-Coder-3B	FS	15.01	8.81
12	GPT-4.1-mini→Qwen2.5-Coder-3B	BP	20.61	13.78
13	GPT-o3-mini→Qwen2.5-Coder-3B	BP	**20.08**	**13.77**

[c.] FS = few-shot, CoT = Chain-of-Thought, BP = basic prompt. N = 1000

B. Key findings

1) Code-specialised SLMs outperform general-purpose SLMs in single-model settings. Among single-model configurations, Qwen2.5-Coder-3B with few-shot prompting achieved the highest OTR at 73.2%, an CTR of 47.4%. Outperforming the best general-purpose model Qwen2.5-3B, with OTR = 45.5% and CTR = 41.1%. This supports second research question, that code-specialized training benefits LLVM-IR transformation tasks.

2) Reasoning prompts (CoT) did not consistently improve single-model results. For some models, adding Chain-of-Though prompting improved CTR, for example Qwen2.5-Coder-1.5B, CTR increase from 10.8% to 20.3%, but in others it reduced OTR, for Qwen2.5-1.5B drop from 67.2% to 33.7%. This indicates that CoT helps tiny models, hurts 3B models, this difference demonstrated in Table IV. Computational overhead and potential distraction introduced by reasoning traces can outweigh the benefits of small, single-model setups.

TABLE IV. EFFECT OF CHAIN-OF-THOUGHT (COT) PROMPTING VS. FEW-SHOT ASELINE FOR SINGLE MODEL CONFIGURATIONS

Model	Metrics	
	ΔCTR (pp)	*ΔOTR (pp)*
Qwen2.5-Coder-1.5B	+9.5	+17.2
Qwen2.5-Coder-3B	-0.3	-12.7
Qwen2.5-3B (Intsr.)	+0.4	-1.8

[d.] Δ values are in percentage point (pp), calculated as CoT-FS

3) Dual-model prompting produces the best overall results. GPT-o3-mini and Qwen2.5-Coder-3B configuration achieved the top OTR (76.4%) and CTR (68.6%), outperforming all single-model baselines. GPT-4.1-mini as reasoner also performed strongly (68.8% OTR and 66.9% CTR). This supports third research question, demonstrating that reasoning generated by a larger model can effectively guide an SLM to higher correctness.

4) Prompt refinement gains. The zero-shot(basic) prompt baseline Qwen2.5-3B and Qwen2.5-Coder-3B scored 66.4% OTR, substantially lower than the same pair with few-shot examples (70.9% OTR and 58.7% CTR). For single-model trials, few-shot prompting serves as the baseline.

5) No-Transformation Rate (NTR) varies by model and strategy. General purpose SLM like Qwen2.5-1.5B (FS) showed high NTR (54.6%), indicates an unwillingness to attempt changes, whereas high-performing dual-model configurations achieved both high CTR and low NTR (GPT-4.1-mini and Qwen2.5-Coder-3B, NRT = 1.9%). Indicating that effective systems attempt transformations more often and with higher semantic accuracy.

6) Distilled reasoning models underperform in this setting. The DeepSeek-R1-Distill-Qwen-1.5B model, used as single-model reasoner, achieved only 16.7% OTR, indicating that aggressive reasoning distillation may degrade performance in SLMs.

7) Instruction count reduction suggests optimization impact. Most verified transformations reduced IR size, with reductions randing from 2.73% to 26.43% among successes. Dual-model pipelines with external reasoning achieved the largest overall gains. For example GPT-o3-mini → Qwen2.5-Coder-3B reduced instructions by 13.77% on average. Full results are in Table III.

C. Summary of Results

The experimental evaluation demonstrates that reasoning-enhanced prompting can improve the semantic correctness of LLVM-IR peephole optimizations performed by SLMs without fine-tuning. Code-specialized models outperformed general-purpose models in single-model setups, with Qwen2.5-Coder-3B (few-shot) achieving **73.2%** OTR and **47.4%** CTR. In dual-model configuration, GPT-o3-mini and Qwen2.5-Coder-3B (few-shot) reached the highest scores (OTR **76.4%** and CTR = **68.6%**), confirming that larger model reasoning can guide SLMs effectively. Beyond correctness, most verified transformations also reduced instruction count. Best pipeline achieving a **20.08%** mean reduction among successes and **13.77%** over all inputs. These results indicate that an increase in correctness is accompanied by an increase in efficiency. Distilled reasoning

model underperformed, highlighting limits of reasoning distillation for this task.

VII. Discussion

A. Alignment with research questions

Explicit reasoning with a larger model improves correctness, with GPT-o3-mini→Qwen2.5-Coder-3B reaching CTR 68.6% and OTR 76.4%, outperforming all single-model baselines (Table II, Fig. 5). In contrast, adding CoT within small models gave mixed results–increasing CTR for Qwen2.5-Coder-1.5B (+9.5 pp) but reducing OTR for Qwen2.5-Coder-3B (–12.7 pp). In single-model mode, code specialized SLMs outperform their instruction-tuned counterparts as Qwen2.5-Coder-3B reaches a CTR of **47.4%,** compared to 41.1-41.5% for Qwen-2.5-3B with few-shot or CoT prompting. Dual model prompting consistently improves correctness. When used as reasoning models, GPT-4.1-mini and GPT-o3-mini deliver higher CTR than small models used for reasoning, indicating that larger-model reasoning can effectively guide small language model generation. It is important to note that as the quality of reasoning provided by the separate model increases, the CTR of the coder model also improves. This modular approach also offers interpretability benefits as reasoning can be examined independently from code-generation results. Importantly, using an open weights model as a reasoner (Qwen2.5-3B) instead of GPT-o3-mini still delivered good results (CTR = **58.7%).** This demonstrates a trade-off between maximum performance (with LLMs) and reproducibility (with open models).

B. Importance of prompt design

Adding better prompts, especially with clear few-shot examples gave an improvement in performance (from 52.3% to 58.7% in the Qwen2.5-3B and Qwen2.5-Coder-3B dual model setup. This shows that even without increasing computing power, well-chosen examples can guide the model effectively. At the same time, the results suggests that such prompting strategies give less benefit for models that are already specialized or fine-tuned for code tasks.

C. Error patterns and the role of CoT

Two patterns emerge. First, CoT helps smaller models but can reduce performance in larger ones. For Qwen2.5-Coder-1.5B CoT increases CTR from 10.8% to 20.3% (+9.5 pp) and OTR from 46.5% to 63.7% (+17.2 pp). For Qwen2.5-Coder-3B, CTR stays almost the same (47.4% and 47.1%) while OTR decreases (73.2% to 60.5%). Second, small model with well-structured reasoning trace provided by larger models has low NTR. For example, GPT-4.1-mini and Qwen2.5-Coder-3B has NTR = 1.9%. Which means the model leaves the IR unchanged in only 1.9% of verified cases indicating that most successful passes involved and actual optimization.

The instruction-count reduction analysis reinforces this trend. In smaller models CoT not only improved CTR but also increased the size reduction. For Qwen2.5-1.5B from 2.73% to 11.09% reduction among successes. In larger single models, CoT slightly reduced mean instruction. Qwen2.5-Coder-3B from 11.09% to 11.07% among successes. While lowering OTR, suggesting that added reasoning steps did not translate into effective optimizations.

DeepSeek-R1-Distill-Qwen-1.5B a distilled reasoning model, shows consistently poor performance (13.8% correctness). This suggest that distillation alone is insufficient to give small models the reliable reasoning abilities required for compiler optimization. Future work could investigate improved distillation techniques.

D. Comparison with related work

The Meta LLM for Compiler Optimization predicts LLVM pass lists instead of generating IR directly, leaving the check to compiler. Our approach addresses the more challenging task of direct IR rewrites. Related work using a fine-tuned LLaMA2-7B model trained on AArch64 peephole optimization achieves approximately 64% semantic correctness. This is included in Fig. 6 as a non-comparable reference. Our best-dual model configuration reaches CTR of 68.6%, exceeding that baseline while using untuned small language model, reducing the total number of parameters by half for generation and relying solely on prompting for control.

E. Limitations

1) Dataset diversity: Although the dataset includes common LLVM IR patterns, it may not reflect the full complexity of large-scale production compilers, which can require deeper reasoning. Evaluation was limited to 1,000 sampled cases due to computational cost, so results may differ on the full dataset.

2) Prompt sensitivity and decoding: Results can vary with exemplars, temperatures and sampling strategies. We did not evaluate self-consistency decoding *(Wang* et al*., 2022)* and Tree-of-Thoughts *(Yao* et al*., 2023)*. This could potentially yield further gains.

3) External reasoners: The most successful configuration required closed-source OpenAI models for reasoning. Using LLMs as reasoners (GPT-4.1-mini and GPT-o3-mini) impacts cost and reproducibility. Although open-weight models like Qwen2.5-3B performed competitively, their CTR was around 10 percentage points lower than GPT-o3-mini. Matching the reasoning quality of closed models with open ones remains an open challenge.

4) Negative results omitted from figures: An attempt to integrate PPO fine-tuning was not successful due to implementation constraints. APE prompt search did not outperform CoT. Descriptions of the use of these methods are provided for completeness, but they are not central to the main results.

5) Sparse semantic feedback: Alive-tv offers only binary correctness feedback, which limits its usefulness for reinforcement learning. Introducing intermediate reward signal, such as partial correctness scores or heuristics checks, could enable more stable and effective training.

6) Context-length constraints: All experiments were performed on a NVIDIA RTX 4090 GPU. Limiting batching and throughput, due to memory constraints.

7) No change bias: Small models have tendency to output the same unoptimized IR in order to pass Alive-tv check. This leads to high NTR. While dual-model configurations reduced this effect, the bias remains a limitation in single-model prompting setups.

8) Reducing the number of instructions is not a direct indicator of performance: Although a reduction in the number of instructions correlates with a smaller and potentially more efficient IR. This study did not measure runtime or memory performance. Thus, the reduction in the number of instructions should be interpreted as an indirect indicator, rather than as an indicator of a real improvement in execution.

F. Conclusion

This study demonstrates that combining external reasoning with small, specialized code-generation models through prompting can deliver reliable and semantically correct compiler optimizations. Despite current limitations, particularly the reliance on external reasoners, a limited dataset, and practical deployment constraints, the results clearly support the effectiveness of reasoning-enhanced prompting strategies and provide a foundation for future work applying language models to compiler optimizations.

G. Further Development

1) Reducing No-change bias: Many models in this study frequently returned the unmodified input IR to secure an Alive-tv pass. To address this, future systems could incorporate explicit loss penalties or reward shaping machanisms to discourage unchanged outputs.

2) Reward-based refinement: Exploring RL variants compatible with non-differentiable rewards to use Alive-tv feedback without a learned reward model.

3) Reasoning ensembles: Evaluate self-consistency and program-of-thoughts on top of the best dual-model setup.

4) Efficient local reasoning: A promising direction is to distill the reasoning ability of large models into smaller local models, training one small model to replicate the dual-model pipeline.

REFERENCES

Abdin, M., et al., 2024. Phi-3 technical report: A highly capable language model locally on your phone. arXiv preprint arXiv:2404.14219. Available at: https://arxiv.org/abs/2404.14219 [Accessed 1 Aug. 2025].

Agarwal, V., Pei, Y., Alamir, S. and Liu, X., 2024. CodeMirage: Hallucinations in code generated by large language models. arXiv preprint arXiv:2408.08333. Available at: https://arxiv.org/abs/2408.08333 [Accessed 31 Jul. 2025].

AliveToolkit, 2025. GitHub – AliveToolkit/alive2: Automatic verification of LLVM optimizations. Available at: https://github.com/AliveToolkit/alive2 [Accessed 3 Aug. 2025].

Anthropic, 2025. Claude 3 model card. Available at: https://www.anthropic.com/claude [Accessed 31 Jul. 2025]

Austin, J., et al., 2021. Program synthesis with large language models. arXiv preprint arXiv:2108.07732. Available at: https://arxiv.org/abs/2108.07732 [Accessed 3 Aug. 2025].

Bhatt, C.H. and Bhadka, H.B., 2013. Peephole optimization technique for analysis and review of compiler design and construction. IOSR Journal of Computer Engineering, (4), pp.80–86. Available at: https://www.iosrjournals.org/iosr-jce/papers/Vol9-Issue4/N0948086.pdf [Accessed 13 Aug. 2025].

Brown, T.B., et al., 2020. Language models are few-shot learners. arXiv preprint arXiv:2005.14165. Available at: https://arxiv.org/abs/2005.14165 [Accessed 5 Aug. 2025].

Chen, W., Ma, X., Wang, X. and Cohen, W.W., 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. arXiv preprint arXiv:2211.12588. Available at: https://arxiv.org/abs/2211.12588 [Accessed 3 Aug. 2025].

Cummins, C., et al., 2023. Large language models for compiler optimization. arXiv preprint arXiv:2309.07062. Available at: https://arxiv.org/abs/2309.07062 [Accessed 31 Jul. 2025].

DeepSeek-AI, 2024. DeepSeek-V3 technical report. arXiv preprint arXiv:2412.19437. Available at: https://arxiv.org/abs/2412.19437 [Accessed 13 Aug. 2025].

DeepSeek-AI, 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948. Available at: https://arxiv.org/abs/2501.12948 [Accessed 3 Aug. 2025].

EvalPlus, 2025. EvalPlus leaderboard. Available at: https://evalplus.github.io/leaderboard.html [Accessed 31 Jul. 2025].

Fang, X. and Mukhanov, L., 2024. Towards LLM-based optimization compilers: Can LLMs learn how to apply a single peephole optimization? Reasoning is all LLMs need! arXiv preprint arXiv:2412.12163. Available at: https://arxiv.org/abs/2412.12163 [Accessed 7 Aug. 2025].

Feng, Z., et al., 2020. CodeBERT: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155. Available at: https://arxiv.org/abs/2002.08155 [Accessed 31 Jul. 2025].

Fried, D., et al., 2022. InCoder: A generative model for code infilling and synthesis. arXiv preprint arXiv:2204.05999. Available at: https://arxiv.org/abs/2204.05999 [Accessed 31 Jul. 2025].

Haj-Ali, A., et al., 2019. AutoPhase: Compiler phase-ordering for high level synthesis with deep reinforcement learning. arXiv preprint arXiv:1901.04615. Available at: https://arxiv.org/abs/1901.04615 [Accessed 4 Aug. 2025].

Havrilla, A., et al., 2024. Teaching large language models to reason with reinforcement learning. arXiv preprint arXiv:2403.04642. Available at: https://arxiv.org/abs/2403.04642 [Accessed 3 Aug. 2025].

Hendrycks, D., et al., 2021. Measuring coding challenge competence with APPS. arXiv preprint arXiv:2105.09938. Available at: https://arxiv.org/abs/2105.09938 [Accessed 3 Aug. 2025].

Hosseini, I. and Dolan-Gavitt, B., 2022. Beyond the C: Retargetable decompilation using neural machine translation. In: Proceedings of the Binary Analysis Research (BAR), pp.133–144. doi:10.14722/bar.2022.23009.

Hugging Face, 2025a. Qwen2.5 – a Qwen collection. Available at: https://huggingface.co/collections/Qwen/qwen25-66e81a666513e518adb90d9e [Accessed 3 Aug. 2025].

Hugging Face, 2025b. Qwen2.5-Coder – a Qwen collection. Available at: https://huggingface.co/collections/Qwen/qwen25-coder-66eaa22e6f99801bf65b0c2f [Accessed 3 Aug. 2025].

Hugging Face, 2025c. Deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B · Hugging Face. Available at: https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B [Accessed 3 Aug. 2025].

Hugging Face, 2025d. PPO trainer. Available at: https://huggingface.co/docs/trl/v0.19.1/en/ppo_trainer [Accessed 6 Aug. 2025].

Kojima, T., Gu, S.S., Reid, M., Matsuo, Y. and Iwasawa, Y., 2022. Large language models are zero-shot reasoners. arXiv preprint arXiv:2205.11916. Available at: https://arxiv.org/abs/2205.11916 [Accessed 1 Aug. 2025].

Lattner, C. and Adve, V., 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In: Proceedings of the International Symposium on Code Generation and Optimization (CGO). Available at: https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf [Accessed 13 Aug. 2025].

Liu, F., et al., 2024. Exploring and evaluating hallucinations in LLM-powered code generation. arXiv preprint arXiv:2404.00971. Available at: https://arxiv.org/abs/2404.00971 [Accessed 10 Aug. 2025].

LLVM.org, 2025. LLVM language reference manual — LLVM 22.0.0git documentation. Available at: https://llvm.org/docs/LangRef.html [Accessed 3 Aug. 2025].

Lopes, N.P., Lee, J., Hur, C-K., Liu, Z. and Regehr, J., 2021. Alive2: Bounded translation validation for LLVM. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), pp.65–79. doi:10.1145/3453483.3454030.

Mankowitz, D.J., et al., 2023. Faster sorting algorithms discovered using deep reinforcement learning. Nature, 618(7964), pp.257–263. doi:10.1038/s41586-023-06004-9.

OpenAI, 2023. GPT-4 technical report. arXiv preprint arXiv:2303.08774. Available at: https://arxiv.org/abs/2303.08774 [Accessed 13 Aug. 2025].

OpenAI, 2025. Introducing GPT-4.1 in the API. Available at: https://openai.com/index/gpt-4-1/ [Accessed 4 Aug. 2025].

Ouyang, L., et al., 2022. Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155. Available at: https://arxiv.org/abs/2203.02155 [Accessed 3 Aug. 2025].

Qwen, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115. Available at: https://arxiv.org/abs/2412.15115 [Accessed 1 Aug. 2025].

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. Available at: https://arxiv.org/abs/1707.06347 [Accessed 3 Aug. 2025].

Shridhar, K., Stolfo, A. and Sachan, M., 2022. Distilling reasoning capabilities into smaller language models. arXiv preprint arXiv:2212.00193. Available at: https://arxiv.org/abs/2212.00193 [Accessed 3 Aug. 2025].

Sun, Z., et al., 2024. Enhancing code generation performance of smaller models by distilling the reasoning ability of LLMs. arXiv preprint arXiv:2403.13271. Available at: https://arxiv.org/abs/2403.13271 [Accessed 10 Aug. 2025].

Touvron, H., et al., 2023. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971. Available at: https://arxiv.org/abs/2302.13971 [Accessed 7 Aug. 2025].

Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K. and Li, D., 2021. MLGO: A machine learning guided compiler optimizations framework. arXiv preprint arXiv:2101.04808. Available at: https://arxiv.org/abs/2101.04808 [Accessed 31 Jul. 2025].

Vaswani, A., et al., 2017. Attention is all you need. arXiv preprint arXiv:1706.03762. Available at: https://arxiv.org/abs/1706.03762 [Accessed 31 Jul. 2025].

VenkataKeerthy, S., Jain, S., Kundu, A., Aggarwal, R., Cohen, A. and Upadrasta, R., 2023. RL4ReAl: Reinforcement learning for register allocation. In: Proceedings of the ACM SIGPLAN Conference on Principles and Practice of Parallel Programming (PPoPP), pp.133–144. doi:10.1145/3578360.3580273.

Wang, X., et al., 2022. Self-consistency improves chain-of-thought reasoning in language models. arXiv preprint arXiv:2203.11171. Available at: https://arxiv.org/abs/2203.11171 [Accessed 3 Aug. 2025].

Wei, J., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903. Available at: https://arxiv.org/abs/2201.11903 [Accessed 31 Jul. 2025].

Yao, S., et al., 2023. Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601. Available at: https://arxiv.org/abs/2305.10601 [Accessed 31 Jul. 2025].

Zhang, J., et al., 2024. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp.11263–11282. doi:10.18653/v1/2024.acl-long.607.

Zhou, Y., et al., 2022. Large language models are human-level prompt engineers. arXiv preprint arXiv:2211.01910. Available at: https://arxiv.org/abs/2211.01910 [Accessed 4 Aug. 2025].

APPENDIX

A – Prompt examples

A.1 Basic prompt

```
system_prompt = (
"You are an advanced LLVM peephole optimizer. Your
sole task is to read LLVM IR code and perform
peephole optimizations "
"on the code. You must output only the optimized
LLVM IR without any commentary, explanation, or
additional formatting. "
"Use LLVM's conventions and transformation rules
(such as constant folding, dead code elimination,
and redundant instruction removal) "
"to produce the most efficient version of the
provided IR."
)

prompt = f"""Below is an unoptimized LLVM IR
snippet. Please perform peephole optimizations on
it and provide the optimized IR output.

Example unoptimized LLVM IR:
{unoptimized_ir}"""
```

A.2 Few-shot prompt

```
few_shot_examples = f"""
Example 1:
Unoptimized LLVM IR
({len(dataset[30001]['non_optimized'].splitlines()
)} lines):
{dataset[30001]['non_optimized']}
Optimized LLVM IR
({len(dataset[30001]['LLVM_instcombine'].splitline
s())} lines):
{dataset[30001]['LLVM_instcombine']}

Example 2:
Unoptimized LLVM IR
({len(dataset[30002]['non_optimized'].splitlines()
)} lines):
{dataset[30002]['non_optimized']}
Optimized LLVM IR
({len(dataset[30002]['LLVM_instcombine'].splitline
s())} lines):
{dataset[30002]['LLVM_instcombine']}

Bad Example:
Unoptimized LLVM IR
({len(dataset[30003]['non_optimized'].splitlines()
)} lines):
{dataset[30003]['non_optimized']}
Optimized LLVM IR
({len(dataset[30003]['non_optimized'].splitlines()
)} lines):
{dataset[30003]['non_optimized']}
"""
prompt = f"""{few_shot_examples}
Now optimize the following LLVM IR (input has
{n_lines} lines; output must have at most
{n_lines-1} lines):
Unoptimized LLVM IR ({n_lines} lines):
{unoptimized_ir}
"""
```

A.3 Chain-of-Thought prompt

```
system_prompt = (
    "You are an LLVM peephole optimizer. "
    "Pick and apply these transformations:
constant folding, dead code elimination, "
    "strength reduction, algebraic simplification,
or redundant load/store removal. "
    "\n\n"
    "You MUST think step by step. First write your
reasoning under the heading '### Plan', "
    "then write the final, optimized LLVM IR under
the heading '### Final IR' enclosed in "
    "a fenced code block: ```llvm … ```. "
    "\n\n"
    "The final IR MUST have strictly fewer lines
than the input and contain no comments or fences."
)
```

A.4 Dual-model prompt

```
REASONER_SYSTEM_PROMPT = (
    "You are an LLVM IR optimization planner.\n"
    "Your ONLY output must be step-by-step
reasoning - one numbered bullet per "
    "peephole transformation that will reduce code
size while preserving semantics. "
    "Do NOT output any LLVM IR - only the
reasoning."
)

CODER_SYSTEM_PROMPT = (
    "You are an advanced LLVM peephole optimiser.
Guided strictly by the given "
    "reasoning steps, transform the provided
unoptimised LLVM IR so that the "
    "result is semantically equivalent but
contains fewer lines.\n"
    "Output ONLY the optimised LLVM IR with NO
commentary."
)
```

A.5 APE generated prompt

```
Eliminate unnecessary memory allocations and loads
by directly using the function parameters and
optimizing arithmetic operations, ensuring that
```

```
the semantics of the original program are
preserved.
```

A.6 Initial experiments with very detailed prompting

```
IR_RULES = r"""
You are an LLVM IR peephole optimizer.
Input: just the module body (everything after the
`target datalayout` / `target triple` lines).
Output: a body that
  • assembles under `llvm-as && opt -verify -S`,
  • passes Alive2 (`alive-tv original.ll
optimised.ll`),
  • and has strictly fewer static instructions
than the input.

0. **Must assemble.** Do not change `define` vs.
`declare`, braces, or syntax.
   If you cannot emit valid IR, return the
original body verbatim.

1. **Preserve definitions.**
   Every function received as `define … { … }`
must stay a `define`.
   Only emit `declare` for symbols not defined in
the input.

2. **Decls for intrinsics.**
   If you add or keep any external/intrinsic
symbol, emit a matching `declare`.
   Remove any now-unused `declare`s.

3. **Header untouched.**
   Do not output or alter the module header—touch
only the body.

4. **Block labels.**
   Label every basic block with a human name (e.g.
`entry`, `loop.latch`);
   never numeric-only labels like `0:`.

5. **SSA renaming.**
   Rename SSA values to `%0`, `%1`, … with no gaps
within each function.

6. **Function attrs.**
   Place attributes after `define … @name(params)`
using only LLVM 14–valid names
   (e.g. `nounwind`, `noundef`, `uwtable`).

7. **Comments.**
   No comment may contain `%` or basic-block
labels.

8. **Memory & UB safety.**
   Do not introduce undefined behavior (`undef`,
`poison`, etc.).
   Reorder loads/stores only when provably
independent.

9. **Loop soundness.**
   Apply only peephole rewrites that are correct
under Alive2's single-loop unroll.

10. **Size reduction.**
    The optimized body must have strictly fewer
static instructions.
    If you cannot prove a safe reduction, output
the original body unchanged.

11. **Output format – reply must be exactly:**
        <think>
        …your internal reasoning…
        </think>
        <llvm>
        …optimised IR body…
        </llvm>
    No markdown, explanations, or extra text
outside those blocks."""
```